



Universidade Federal do Rio Grande do Norte
Centro de Tecnologia - CT
Departamento de Engenharia Elétrica - DEE
Laboratório de Automação, Controle e Instrumentação - LACI

Máquina de Troco

ELE3515 - Grupo 01 - Problema 03 - Projeto

Líder:	Nicolas Pinheiro Silva
Redator:	Karen Helloisa Araújo Silva
Debatedor:	Rinaldo Tavares da Silva Filho

Natal, 14 de outubro de 2025

Resumo

Este trabalho apresenta o projeto e a implementação de uma máquina de troco inteligente na disciplina de Sistemas Digitais. O sistema foi desenvolvido para calcular trocos de forma automática, utilizando a metodologia de projeto RTL.

A implementação envolveu elementos combinacionais, como multiplexadores e demultiplexadores, e elementos de memória, como registradores, para armazenamento e processamento dos dados. A arquitetura foi organizada em módulos funcionais, incluindo registradores para o valor total da subtração, estado da máquina e quantidade de moedas em cada cofre. Cada módulo foi projetado, simulado e validado individualmente antes da integração ao sistema completo.

Os resultados confirmaram o funcionamento correto das operações propostas, evidenciando a viabilidade do projeto e proporcionando uma aplicação prática dos conceitos de circuitos digitais.

Palavras-chave: Máquina de troco; Sistemas Digitais; Projeto RTL.

Sumário

1	Introdução	4
2	Fundamentação Teórica	6
2.1	Metodologia de Projeto RTL	6
2.2	Demultiplexadores	8
3	Desenvolvimento da Solução	9
3.1	Botão Síncrono	9
3.2	Máquina de troco	11
3.2.1	Bloco Operacional	12
3.2.2	Bloco de Controle	15
3.3	Ligação do Operacional com o Controle	17
4	Conclusão	20
A	Relato semanal	22
A.1	Equipe	22
A.2	Defina o problema	22
A.3	Pontos-chaves	22
A.4	Questões de pesquisa	22
A.5	Planejamento da pesquisa	22
B	Anexos	23

1 Introdução

Como já antes visto, nos circuitos sequenciais as saídas não são simplesmente determinadas pela combinação lógica das entradas, mas por uma combinação de entradas com o estado anterior do circuito, determinada por uma Máquina de Estados Finitos (FSM). Uma FSM é o conjunto de estados que representa todos os estados de um sistema, desse modo, representam o funcionamento de circuitos sequenciais. Isso acontece pois em um circuito digital, os registradores armazenam bits, e bits armazenados significam que o circuito tem memória, o que também é conhecido como estado, que resultam os circuitos sequenciais com estados. Assim, podemos usar esses estados para projetar circuitos que tenham determinados comportamentos ao longo do tempo (Vahid 2008).

Dessa forma, a utilização de uma FSM torna-se essencial no projeto da máquina de troco, pois esse sistema requer a execução de diversas etapas de forma sequencial e controlada. Cada estado da FSM representa uma fase específica do funcionamento, como o carregamento do valor de entrada, a verificação dos cofres disponíveis, a subtração progressiva do valor total e liberação das moedas correspondentes. Assim, a FSM permite que o circuito altere seu comportamento ao longo do tempo conforme o estado atual e as condições de entradas, garantindo que o processo de troco ocorra de maneira ordenada, lógica e sincronizada com o clock do sistema.

O problema proposto consistiu no desenvolvimento e implementação de uma máquina de troco com os seguintes sinais de entradas e saídas e suas respectivas funções representadas na Tabela 1.1.

Sinais	Pinos	Funções
Entrada T	KEY[3]	Botão sincronizado
Entrada V	SW[0:9]	Entrada do troco do cliente
Entrada c	SW[10:15]	Indicação de cofre vazio
Saída i	LEDR[1:6]	Leds que indicam a liberação de cada moeda
Saída L	LEDG[0]	LED que indica processamento do troco
Entrada reset	KEY[0]	Reset no sistema
Entrada clk	Clock27	Pulso de clock

Tabela 1.1: Sinais de Entradas e Saídas da Máquina de Cofre

A máquina de troco libera em moedas um valor determinado colocado em sua entrada. A liberação das moedas é realizada por um sistema cofre que libera uma moeda sempre que em sua entrada ix existir um nível lógico alto e ocorrer um pulso de clock. A entrada com o valor do troco a ser liberado é realizada através de um número binário V e do pulso gerado a partir da saída do circuito de um botão sincronizado (BS)

cuja entrada é T. Adicionalmente, a máquina possui uma saída L, a qual quando está piscando indica que a máquina está processando a informação para liberar o troco e qualquer outra solicitação de troco será ignorada. A máquina de troco possui ainda a capacidade de verificar se algum dos cofres de moedas está vazio ($cx = 0$) e recalcula o troco para liberar moedas apenas dos cofres que não estão vazios. Por fim, a máquina de troco indicará que não consegue trocar o valor da entrada V mantendo a saída L em nível lógico alto até que um novo valor do troco a ser liberado seja carregado na máquina.

2 Fundamentação Teórica

2.1 Metodologia de Projeto RTL

O projeto em nível de transferência entre registradores, ou projeto RTL, consiste em uma ampla variedade de abordagens. No entanto, geralmente um projetista especifica os registradores de um circuito, descreve as possíveis transferências e operações a serem realizadas com os dados de entrada, de saída e dos registradores, e define o controle que especifica quando transferir e operar com dados.

Para iniciar um projeto em RTL, é importante a criação de uma máquina de estados de alto nível cuja as condições para as transições e as ações dos estados são mais do que operações booleanas envolvendo os bits de entrada e de saída. São necessários 4 passos para projetar um circuito sequencial em RTL, conforma a Tabela 2.1.

Ordem	Etapa	Descrição
1	Obtenha uma máquina de estados de alto nível	Descreva o comportamento desejado do sistema na forma de uma máquina de estados de alto nível, consistindo em estados e transições.
2	Crie um bloco operacional	Partindo da máquina de estados de alto nível, crie um bloco operacional capaz de realizar as operações que envolvem dados.
3	Conecte o bloco operacional a um bloco de controle	Conecte o bloco operacional a um bloco de controle. Conecte também as entradas e saídas booleanas que são externas ao bloco de controle.
4	Obtenha a FSM do bloco de controle	Converta a máquina de estados de alto nível na máquina de estados finitos do bloco de controle. Para isso, substitua as operações que envolvem dados por sinais de controle, que são ativados ou lidos pelo bloco de controle.

Tabela 2.1: Metodologia de projeto de RTL

O bloco que define um circuito sequencial conforme o projeto RTL está ilustrado na Figura 2.1.

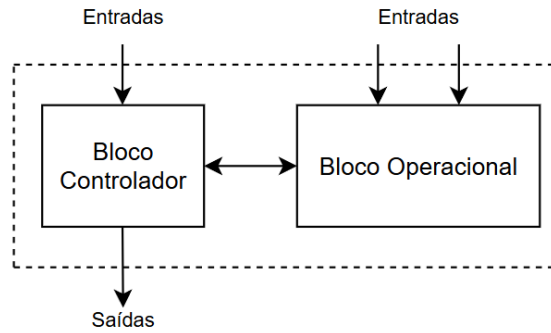


Figura 2.1: Circuito sequencial com RTL

O bloco de controle é essencial em um circuito sequencial projetado em nível de transferência entre registradores. Ele é responsável por coordenar o fluxo de dados dentro do sistema, determinando quando e como os dados devem ser transferidos entre registradores e outras unidades funcionais, como somadores, comparadores, multiplexadores, etc. O bloco de controle é um circuito sequencial que controla saídas booleanas com base em entradas booleanas e em um comportamento específico, ordenado no tempo. Para construir tal bloco, é necessário um registrador de estado, que armazena o estado atual da máquina de controle, e uma lógica combinacional, que determina o próximo estado com base nas entradas atuais e no estado armazenado, e as saídas de controle, que comandam transferências de dados, ativações de módulos ou sinais de alimentação (Vahid 2008).

São necessários 5 passos para projetar um bloco controlador, conforme a Tabela 2.2.

Ordem	Etapa	Descrição
1	Capture a FSM	Crie uma FSM que descreva o comportamento desejado do bloco de controle.
2	Crie a arquitetura	Crie a arquitetura usando um registrador de estado com largura apropriada e uma lógica combinacional, cujas entradas são os bits do registrador de estado e as entradas da FSM e cujas saídas são os bits de próximo estado e as saídas da FSM.
3	Codifique os estados	Atribua um número binário único a cada um dos estados.
4	Crie a tabela de estados	Crie uma tabela-verdade para a lógica combinacional de modo tal que a lógica irá gerar as saídas e os sinais de próximo estado corretos para a FSM.
5	Implemente a lógica combinacional	Implemente a lógica combinacional usando qualquer método.

Tabela 2.2: Metodologia de projeto de bloco de controle

2.2 Demultiplexadores

Demultiplexadores são componentes que recebem uma única entrada de dados e a distribuem para uma de várias saídas, de acordo com os sinais de seleção. Também podem ser chamados de distribuidores, pois direcionam o valor de entrada para a saída selecionada. Este controle é feito por meio de entradas de seleção.

Um demultiplexador de 1 entrada e 2 saídas, conhecido como DEMUX 1x2, possui uma entrada de dados d , uma entrada de seleção s_0 e duas saídas y_0 e y_1 . Quando $s_0 = 0$, o valor de d é direcionado para y_0 e y_1 permanece em 0. Quando $s_0 = 1$, o valor de d é direcionado para y_1 e y_0 permanece em 0. A tabela-verdade que representa este componente está ilustrada abaixo.

Tabela 2.3: Tabela-verdade de um DEMUX 1x2

s_0	d	y_0	y_1
0	1	1	0
0	0	0	0
1	1	0	1
1	0	0	0

As expressões lógicas das saídas y_0 e y_1 são dadas por:

$$y_0 = d \cdot s_0' \quad \text{e} \quad y_1 = d \cdot s_0$$

Para demultiplexadores de ordem superiores é possível fazer o empilhamento de demultiplexador de ordem baixa, conforme a Fig.2.2

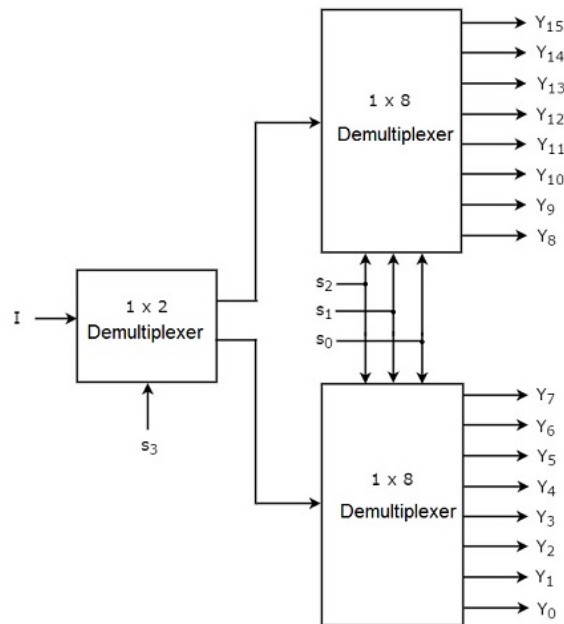


Figura 2.2: Demux 1x16 Fonte: (TP 2025)

3 Desenvolvimento da Solução

3.1 Botão Síncrono

O botão síncrono T foi projetado de modo que quando o botão for apertado, o resultado será um sinal que fica em nível alto por um ciclo de relógio. Ele é útil para evitar que um simples aperto de botão, que pode durar diversos ciclos de relógio, seja interpretado como múltiplos apertos. Este sistema tem entrada bi , representando o aperto no botão, e saída $b0$. A FSM inicial foi projetada e, posteriormente, os estados $S0$, $S1$ e $S2$ foram respectivamente codificados pelos números binários 00, 01 e 10, conforme ilustrado na Figura 3.1.

O funcionamento desta máquina de estados acontece por meio das seguintes etapas:

- 1) A FSM fica esperando no estado $S0$ gerando uma saída $b0=0$ até que bi seja 1;
- 2) Quando $bi=1$, a FSM faz a transição para o estado $S1$, produzindo a saída $b0=1$.
- 3) A seguir, a FSM faz uma transição para o estado $S0$ ou $S2$, os quais ambos geram $b0=0$ novamente. Desse modo, $b0$ foi 1 por apenas um ciclo. Se bi retornar a 0, a FSM passará de $S1$ para $S0$. Se bi ainda for 1, a FSM irá para o estado $S2$, no qual fica esperando que bi retorne a 0, causando uma transição de volta ao estado $S0$.

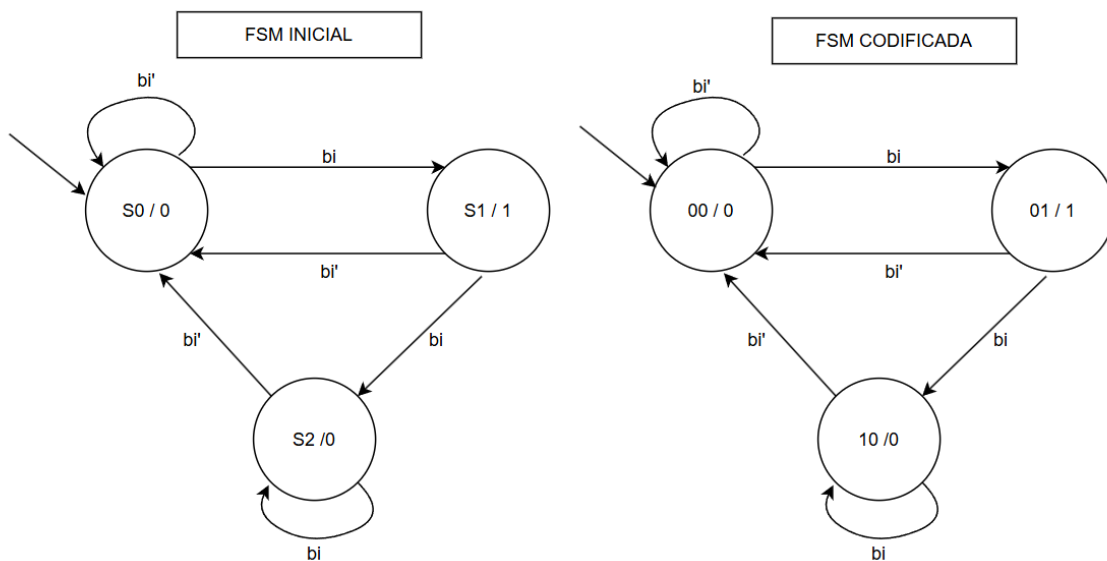
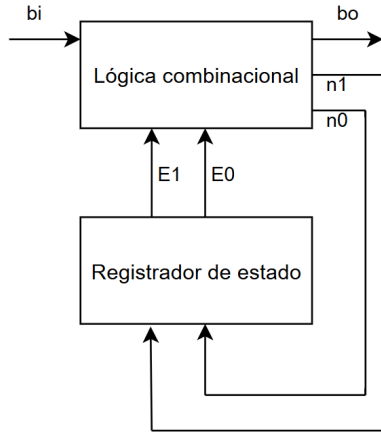


Figura 3.1: FSM inicial e codificada do botão síncrono

Desse modo, concluímos que A FSM tem 3 estados que podem ser representados por 2 bits. Os estados atuais são denominados $E1$ e $E0$ e o estados futuros são denominados

$n1$ e $n0$. Assim, são necessários 2 registradores para armazenar estes estados.

A arquitetura desse sistema é descrita de acordo com a Figura 3.2 e a tabela-verdade que contém a lógica combinacional dos estados é descrita na Tabela 3.1.



E1	E0	bi	n1	n0	b0
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	0	1
1	0	0	0	0	0
1	0	1	1	0	0
1	1	x	x	x	x

Tabela 3.1: Tabela-verdade do botão síncrono

Figura 3.2: Arquitetura do Botão Síncrono

No qual, as expressões lógicas por cada saída são:

$$b0 = E0$$

$$n0 = E1' \cdot E0' \cdot bi$$

$$n1 = E1 \cdot bi + E0 \cdot bi$$

O circuito lógico está ilustrado na Figura 3.3.

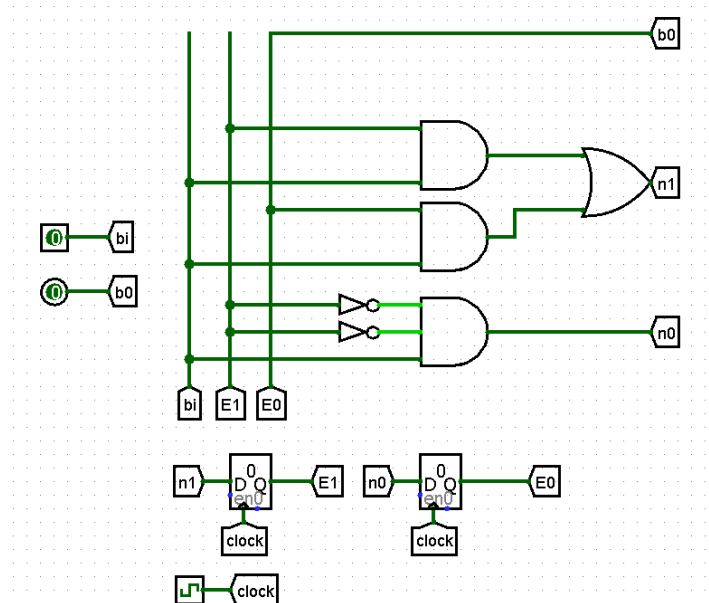


Figura 3.3: Circuito lógico do Botão síncrono

3.2 Máquina de troco

A máquina de Troco foi inicialmente desenvolvida conforme a Máquina de Estados de Alto Nível, de apenas 2 bits e 4 estados, visando simplificar a abstração do problema. Entretanto, esta simplificação do número de estados gerou uma maior complexidade no bloco operacional, conforme será discutido posteriormente. A FSM está apresentada na Figura 3.4.

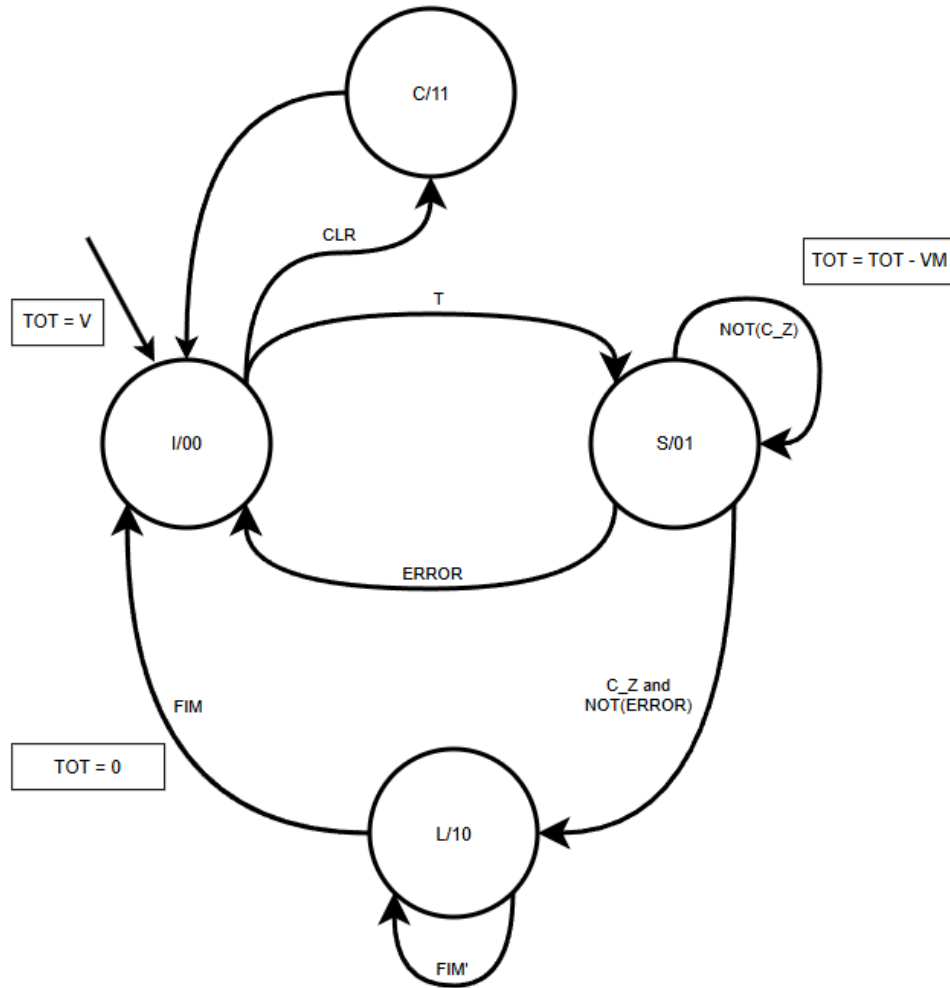


Figura 3.4: Máquina de Estado de Alto Nível

O sistema inicia no estado I (Estado de Início), no qual o valor binário de 10 bits, correspondente a valores entre 0 e 1000 (R\$0,00 a R\$10,00), é carregado no registrador TOT. Esse registrador é responsável por armazenar o valor que será posteriormente operado pelo subtrator. Nesse estado, a máquina pode transitar para dois outros estados: o estado C (Estado de Clear) ou o estado S (Estado de Subtração).

O estado C é responsável por redefinir os valores iniciais dos registradores correspondentes aos cofres de moedas, garantindo a inicialização adequada do sistema.

Por sua vez, o estado S é encarregado de subtrair valores do total armazenado, conforme a disponibilidade de moedas no cofre. A cada operação de subtração, segue a seguinte lógica:

- Subtrair do total o valor da moeda de maior valor possível;
- Subtrair uma unidade do cofre de moedas correspondente;
- Somar uma unidade no cofre de liberação;
- Comparar o valor restante da subtração com zero, gerando o sinal **C_Z**;
- Gerar o sinal **ERROR** caso o valor a ser subtraído seja maior que zero e não haja moedas suficientes no cofre correspondente.

Caso o sistema esteja no estado S e receba o sinal **C_Z** em nível lógico HIGH, independente do **ERROR**, ele irá para o estado L (Estado de Liberação). Por outro lado, caso receba o sinal **ERROR** em nível lógico HIGH, o sistema retornará ao estado I, indicando que não há troco suficiente disponível nos cofres.

O estado L (Estado de Liberação) é responsável pela liberação das moedas. A cada ciclo de operação, uma moeda é liberada de cada cofre que o valor seja superior a zero. Simultaneamente, a cada liberação, é subtraída uma unidade do cofre de liberação correspondente, garantindo a atualização correta do saldo de moedas disponíveis.

Quando todos os cofres estão vazios o circuito gera um sinal **FIM HIGH**, e o circuito volta para o estado I.

3.2.1 Bloco Operacional

O bloco operacional está ilustrado na Figura 3.5. Ele possui duas entradas principais: **S_1**, um sinal de 2 bits que indica o estado atual da máquina, e o valor V, correspondente à entrada de 10 bits. Ele apresenta três saídas: os sinais **ERROR**, **C_Z** e o sinal **FIM** proveniente do bloco Liberador.

Dentro do bloco operacional há três conjuntos de registradores, descritos a seguir:

- Registrador TOT: responsável por armazenar o valor total a ser operado pelo sistema;
- Registradores do cofre de moedas: conjunto de seis registradores de 3 bits, onde são armazenados os valores atuais de cada cofre de moedas;
- Registradores do cofre de liberação: conjunto de seis registradores de 3 bits, responsáveis por armazenar os valores referentes aos cofres de liberação.

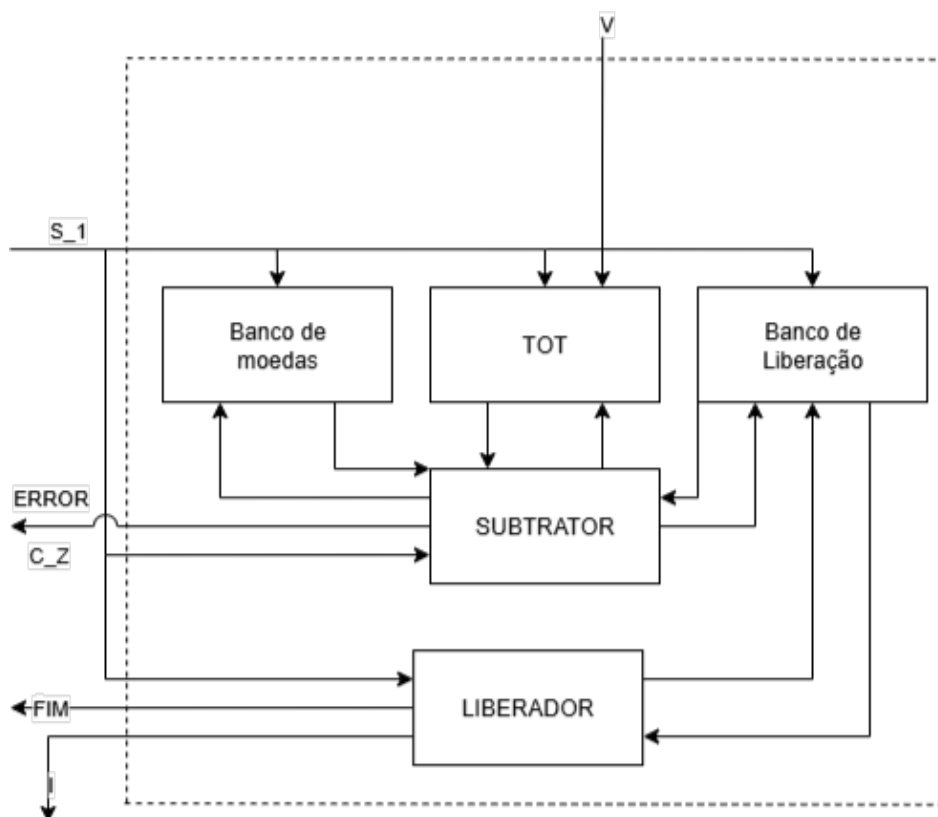


Figura 3.5: Bloco Operacional

Além desses componentes, o bloco operacional também é composto pelos módulos SUBTRATOR e LIBERADOR, responsáveis, respectivamente, pelas operações de subtração e de liberação de moedas.

Subtração

No bloco de SUBTRAÇÃO, as entradas consistem nos valores dos cofres de moedas, cofres de liberação, o valor total (TOT) e o sinal **ENABLE**. O circuito realiza a comparação de cada cofre de moedas individualmente com zero, verificando também se o valor de cada moeda é menor ou igual ao valor total armazenado em TOT. Caso o valor total seja maior ou igual ao valor da moeda correspondente, o sinal associado àquela moeda é definido como HIGH, gerando assim os sinais **C1** a **C6**.

Com os sinais **C1** a **C6** e o **ENABLE** é possível decidir o passo da subtração, se há **ERROR**, e verificar o cofre de moedas que deverá ser subtraído uma unidade e o cofre de liberação que deverá ser somado uma unidade. Essa lógica está descrita na Tabela 3.2.

O passo é dado em binário, sendo o maior valor 100 em decimal (1100100 em binário). Com essa lógica a moeda selecionada para subtração sempre será a maior possível para que não gere valor negativo na subtração. As saídas M1-M6 e C correspondem ao cofre das moedas que será subtraído/somado. Caso o valor seja LOW o cofre das moedas será subtraído/somado, caso seja HIGH o valor é mantido.

Tabela 3.2: Tabela de Verdade Passo

Entradas							Saídas								
C1	C2	C3	C4	C5	C6	ENABLE	C[2:0]	PASSO	ERROR	M1	M2	M3	M4	M5	M6
-	-	-	-	-	-	0	111	0000000	0	1	1	1	1	1	1
0	0	0	0	0	0	1	111	0000000	1	1	1	1	1	1	1
0	0	0	0	0	1	1	101	0000001	0	1	1	1	1	1	0
0	0	0	0	1	-	1	100	0000101	0	1	1	1	1	0	1
0	0	0	1	-	-	1	011	0001010	0	1	1	1	0	1	1
0	0	1	-	-	-	1	010	0011001	0	1	1	0	1	1	1
0	1	-	-	-	-	1	001	0110010	0	1	0	1	1	1	1
1	-	-	-	-	-	1	000	1100100	0	0	1	1	1	1	1

O sinal de **ERROR** só ocorre quando não há mais troco, ou seja ou todos os cofres de moedas estão vazias, ou as moedas atuais não conseguem realizar o troco. O sinal de **C_Z** é HIGH quando a subtração é negativo ou igual a zero.

Como o sistema não opera em sinais negativos é necessário corrigir nesses casos, senão é possível que moedas sejam erroneamente operadas. Para corrigir este caso foi usado um MUX que é controlado pelo carry_out do subtrator, caso esse sinal seja HIGH é gravado o valor 0 na saída do TOT, caso LOW é gravado o valor de TOT - PASSO.

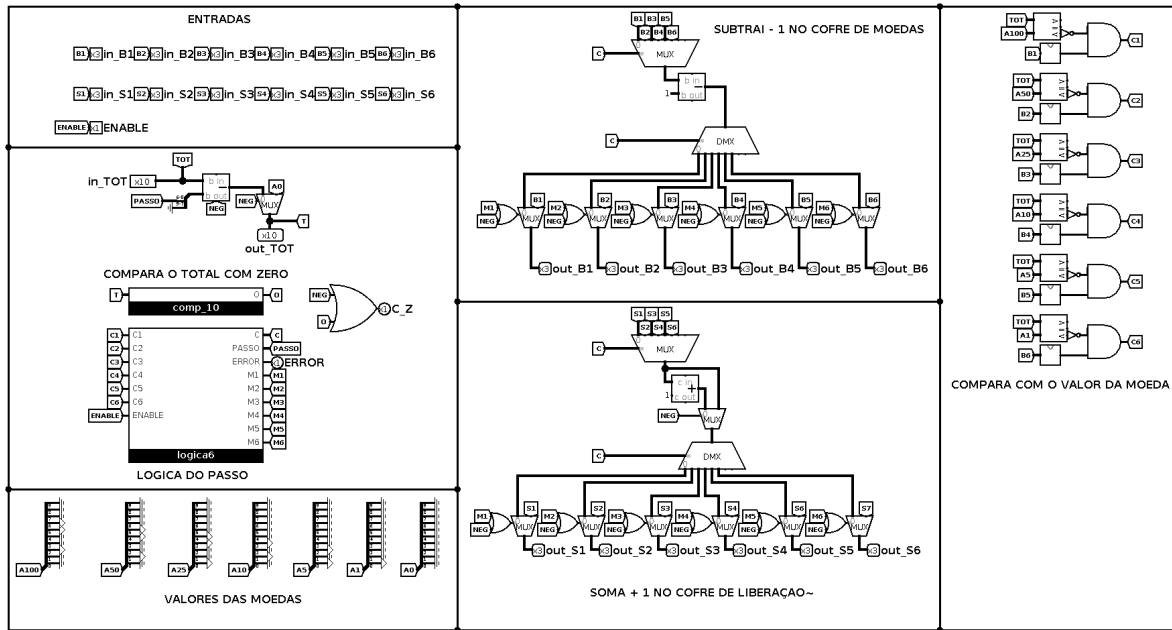


Figura 3.6: Circuito Subtrator

Nos circuitos de subtrações e adições dos cofres de moedas e liberação respectivamente é realizado por um arranjo de um MUX e um DEMUX, com um subtrator e um somador de 1 unidade no meio. Após esse arranjo há um MUX extra que define se o valor é mantido ou subtraído conforme a Figura B.1.

Liberação

O bloco de liberação tem como objetivo realizar a liberação das moedas acumuladas nos cofres de liberação. Para isso, a cada operação, cada cofre é individualmente

comparado com zero por um AND, caso seja diferente de zero o seu valor é reduzido uma unidade por um subtrator e um sinal **I1** a **I6** correspondente ao cofre é posto no estado HIGH, caso seja igual a zero o valor é mantido e o sinal **I1-I6** é posto em LOW, conforme Figura B.2.

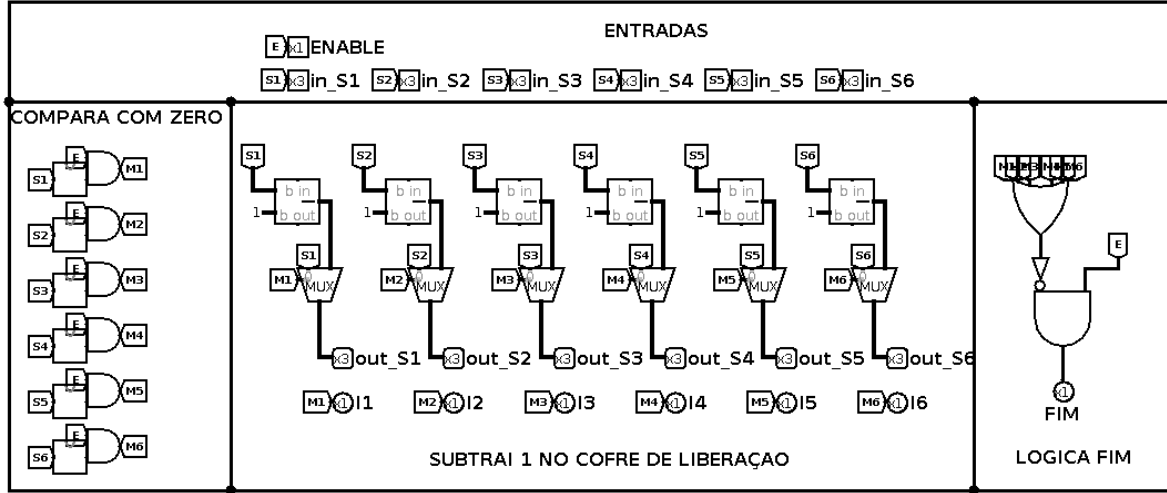


Figura 3.7: Circuito Liberador

O sinal **FIM** só sera posto em HIGH caso todos os cofres de liberação estejam vazios e o **ENABLE** esteja em HIGH.

3.2.2 Bloco de Controle

O bloco de controle é responsável por mudar os estados do bloco de operacional, ou seja no estado de $C_Z = 00$, o bloco operacional apenas grava o valor de entrada V no registrador TOT . Quando no estado 01 irá operar usando o bloco $SUBTRATOR$, quando no estado 10 irá operar usando o bloco $LIBERADOR$, e quando no estado 11 irá realizar o clear dos cofres de moedas e a liberação delas.

O bloco de controle opera a cada clock, e sempre é atualizado o valor do estado atual nos registradores de estado, nesse caso de 2 bits. Para decidir qual estado é o próximo (S_1), é feita uma relação entre o estado do clock anterior (S_0) e as entradas S_0 , $ERROR$, C_Z , FIM , T e CLR .

A tabela-verdade do bloco de controle segue a Tabela 3.3.

Tabela 3.3: Tabela da Verdade dos Sinais de Controle

ERROR	C_Z	FIM	T	S ₀ [1..0]	CLR	S ₁ [1..0]
-	-	-	0	0	0	0 0
-	-	-	0	0	1	1 1
0	0	-	-	0	-	0 1
-	-	0	-	1	-	1 0
-	-	-	0	1	0	0 0
-	-	-	0	1	1	1 1
-	-	-	1	0	-	0 1
-	-	-	1	1	-	0 1
-	-	1	-	1	-	0 0
-	1	-	-	0	1	1 0
1	0	-	-	0	1	0 0

Seguindo a lógica, caso a máquina esteja no estado 00, ela só irá para o estado 01 se o sinal T, do botão, estiver em HIGH, independentemente dos outros sinais. Irá para o estado 11 caso o sinal CLR esteja em HIGH e T em LOW.

No estado 01, a máquina só irá transitar para o estado 00 caso o sinal ERROR esteja em HIGH e C_Z em LOW, indicando que não há troco suficiente. Irá para o estado 10 caso C_Z esteja em HIGH, independentemente dos outros sinais.

No estado 10, a máquina só pode ir para 00; isso ocorrerá quando o sinal FIM estiver em HIGH, independentemente dos outros sinais.

No estado 11, a máquina só poderá transitar para o estado 00, e isso ocorrerá quando C_Z estiver em LOW.

Resolvendo a tabela verdade 3.3 temos as seguintes expressões:

- $S_1(1) = [C_Z + S_0(1) + \overline{S_0(0)}] * [\overline{FIM} + S_0(1) + S_0(0)] * [S_0(1) + S_0(0) + \overline{CLR}] * [\overline{S_0(1)} + S_0(0) + \overline{CLR}] * [\overline{T} + S_0(1) + S_0(0)] * [T + S_0(1) + S_0(0)]$
- $S_1(0) = [T + S_0(0) + \overline{CLR}] * [\overline{S_0(1)} + S_0(0)] * [T + \overline{S_0(1)} + \overline{CLR}] * [\overline{C_Z} + S_0(1) + S_0(0)] * [\overline{ERROR} + S_0(1) + S_0(0)]$

O circuito implementado segue conforme a Figura 3.8.

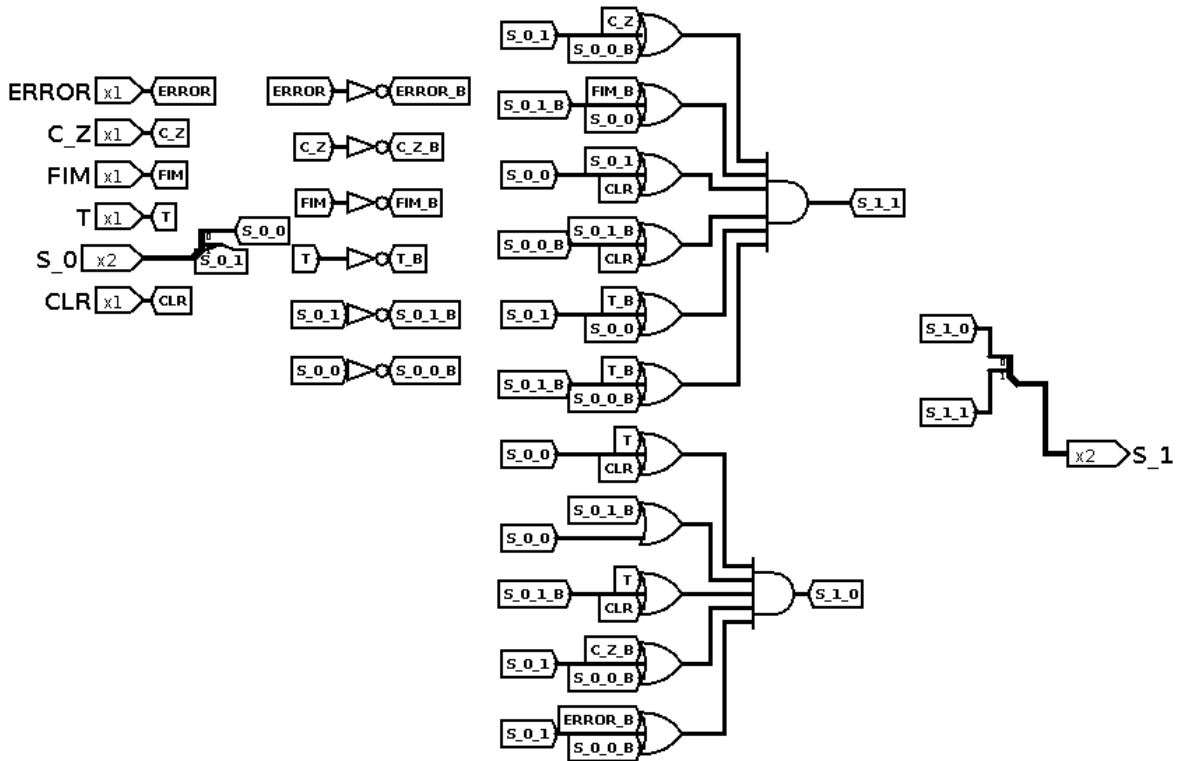


Figura 3.8: Circuito da lógica de controle

Como é necessário armazenar o valor do estado atual, para o próximo ciclo foi utilizado um registrador de 2 bits. Desse modo, a montagem do circuito segue a Figura 3.9.

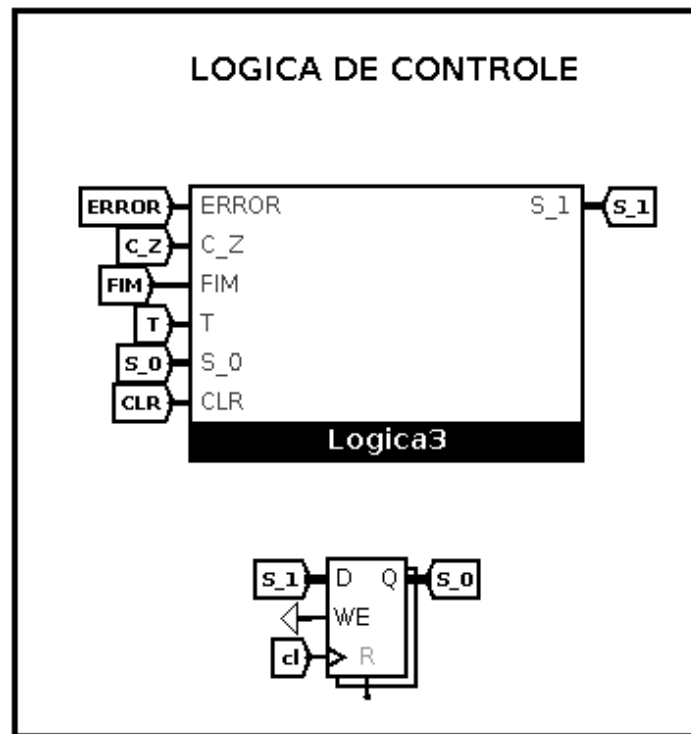


Figura 3.9: Bloco de Controle

3.3 Ligação do Operacional com o Controle

Conforme discutido anteriormente é necessário conectar o bloco de operação com o bloco de controle, seguindo a lógica da Figura 3.5 é possível conectar os dois blocos, a Figura B.3 mostra a montagem do circuito, onde o bloco de controle e bloco de operação são conectados.

Os registradores de moedas e de liberação são ambos de 3 bits com valor máximo de 7 em decimal, assim é possível que as moedas do cofre de moedas se esgotem, podendo testar a logica da maquina de troco completamente. O registrador TOT é de 10 bits. Os valores gravados nos registradores depende do valor do estado atual S_0, seguindo a seguinte logica:

Cofre de moedas (6 registradores de 3 bits)

- S_0 (00) Não altera;
- S_0 (01) Grava o valor que sai do bloco de subtração;
- S_0 (10) Não altera;
- S_0 (11) Grava o valor de clear (7);

Cofre de liberação (6 registradores de 3 bits)

- S_0 (00) Não altera;

- S_0 (01) Grava o valor que sai do bloco de subtração;
- S_0 (10) Grava o valor que sai do bloco de liberação;
- S_O (11) Grava o valor de clear (0);

Registrador de TOT (1 registrador de 10 bits)

- S_0 (00) Grava o valor de V (valor de entrada);
- S_0 (01) Grava o valor que sai do bloco de subtração;
- S_0 (10) Não altera;
- S_O (11) Não altera;

A conexão entre os blocos operacionais e de controle estão ilustrados na Figura 3.10.

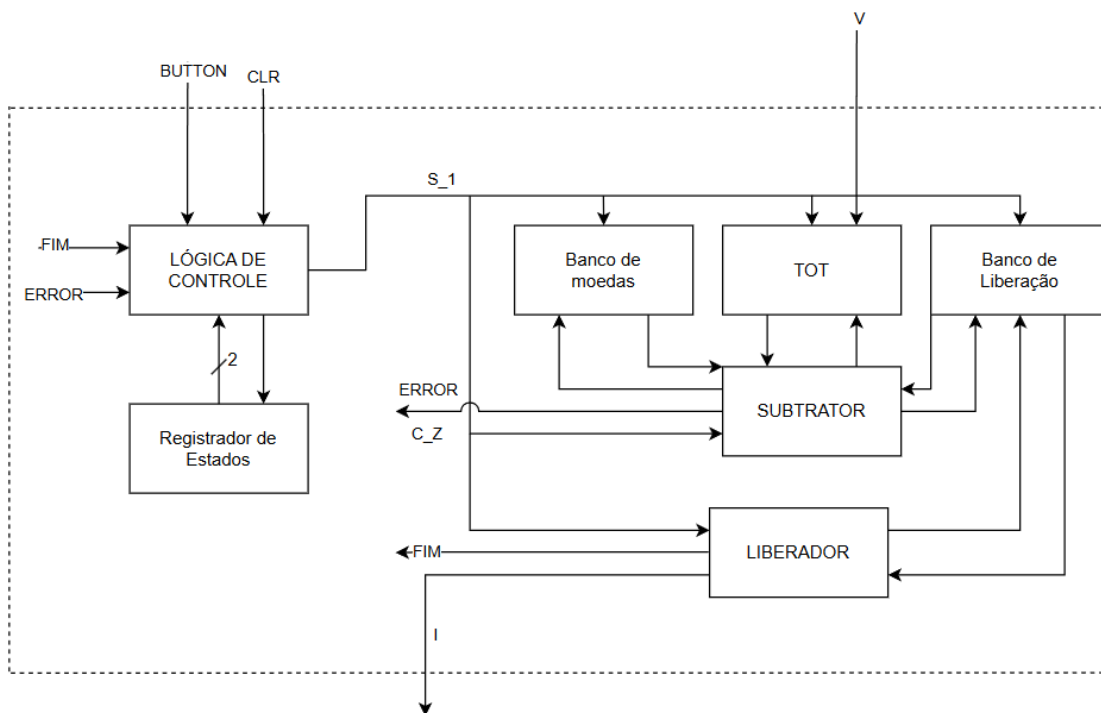


Figura 3.10: Bloco Operacional e de Controle

Além disso, como é requisito de projeto gerar um sinal HIGH de **C1** a **C6** quando os cofres estão vazios foi adicionado um comparador com zero em todos os cofres de moedas. Assim a montagem fica a seguinte:

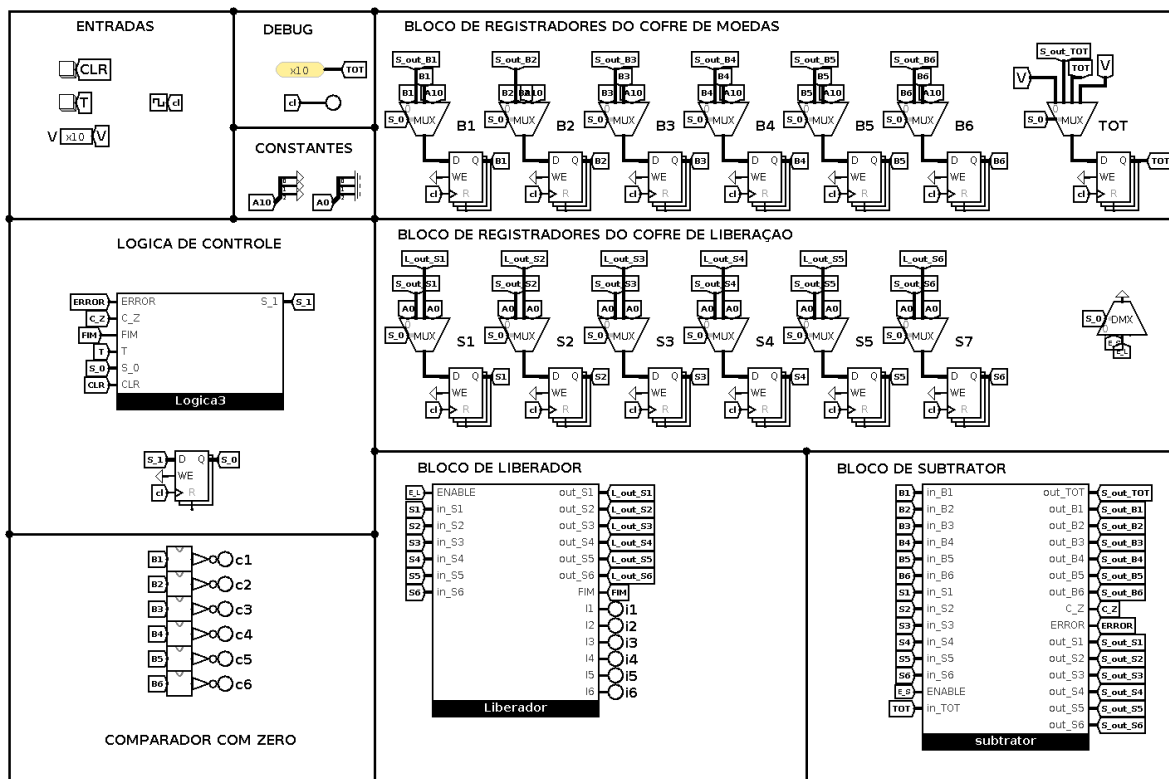


Figura 3.11: Máquina de troco

4 Conclusão

O desenvolvimento da máquina de troco permitiu consolidar os conhecimentos teóricos da disciplina de Sistemas Digitais por meio de uma aplicação prática voltada ao projeto RTL. O sistema foi estruturado em blocos, incluindo registradores para armazenar valores de moedas, total acumulado e troco a ser fornecido, além da unidade de controle principal, o que facilitou a validação de cada bloco e a integração em um sistema completo.

Foram utilizados elementos combinacionais, como somadores e multiplexadores, e sequenciais, como Flip-Flops do tipo D. Além disso, foi aplicado o projeto RTL desde uma máquina de estado de alto nível até a simulação no software Logisim.

Os resultados obtidos confirmaram que a máquina de troco atendeu plenamente aos requisitos estabelecidos, sendo capaz de receber diferentes valores de moedas, calcular automaticamente o troco e liberar a quantidade correta.

Referências Bibliográficas

TP 2025 TP. *Digital Electronics - Demultiplexers*. 2025. Acesso em: 13 out. 2025. Disponível em: <<https://www.tutorialspoint.com/digital-electronics/digital-electronics-demultiplexers.htm>>.

Vahid 2008 VAHID, F. *Sistemas digitais: projeto, otimização e HDLs*. Porto Alegre: Artmed, 2008. 560p; 25 cm. ISBN 978-85-7780-190-9.

A Relato semanal

A.1 Equipe

Função no grupo	Nome completo do aluno	Responsável por
Líder:	Nicolas Pinheiro Silva	Funções: Líder.
Redator:	Karen Helloisa Araújo Silva	Função: Relatório.
Debatedor	Rinaldo Tavares da Silva Filho	Funções: Relatório.

A.2 Defina o problema

Projeto de um maquina de troco.

A.3 Pontos-chaves

Projeto RTL, maquina de estados de alto nível;

A.4 Questões de pesquisa

Projetos RTL, maquina de estados de alto nível;

A.5 Planejamento da pesquisa

([Vahid 2008](#)) apenas.

B Anexos

LOGICA DO PASSO

- $C_2 = \overline{C1} \overline{C2} \overline{C3} C4 + \text{ENABLE}$
- $C_1 = \overline{C1} C2 \overline{C5} \overline{C6} + \text{ENABLE} + C1 \overline{C2} C4 + \overline{C1} C2 C3$
- $C_0 = C1 C3 \overline{C5} + \text{ENABLE} + C1 \overline{C3} C4 + \overline{C1} C2$
- $PASSO_6 = C1 \text{ENABLE}$
- $PASSO_5 = C1 \overline{C2} \text{ENABLE}$
- $PASSO_4 = (\overline{C1} C2 + C1 C3) \text{ENABLE}$
- $PASSO_3 = (\overline{C1} \overline{C2} C3 + \overline{C1} C2 C4) \text{ENABLE}$
- $PASSO_2 = (C1 + C2 C3 \overline{C4} C5) \text{ENABLE}$
- $PASSO_1 = (C1 \overline{C2} + C1 \overline{C3} C4) \text{ENABLE}$
- $PASSO_0 = (\overline{C1} \overline{C2} \overline{C3} + C1 C2 \overline{C4} C6 + \overline{C1} C2 \overline{C4} C5) \text{ENABLE}$
- $ERROR = \overline{C1} C2 C3 C4 C5 C6 \text{ENABLE}$
- $M_1 = C1 + \text{ENABLE}$
- $M_2 = \overline{C1} + C2 + \text{ENABLE}$
- $M_3 = \overline{C1} + C2 + C3 + \text{ENABLE}$
- $M_4 = \overline{C1} + C2 + C3 + C4 + \text{ENABLE}$
- $M_5 = \overline{C1} + C2 + C3 + C4 + C5 + \text{ENABLE}$
- $M_6 = \overline{C1} + C2 + C3 + C4 + C5 + C6 + \text{ENABLE}$

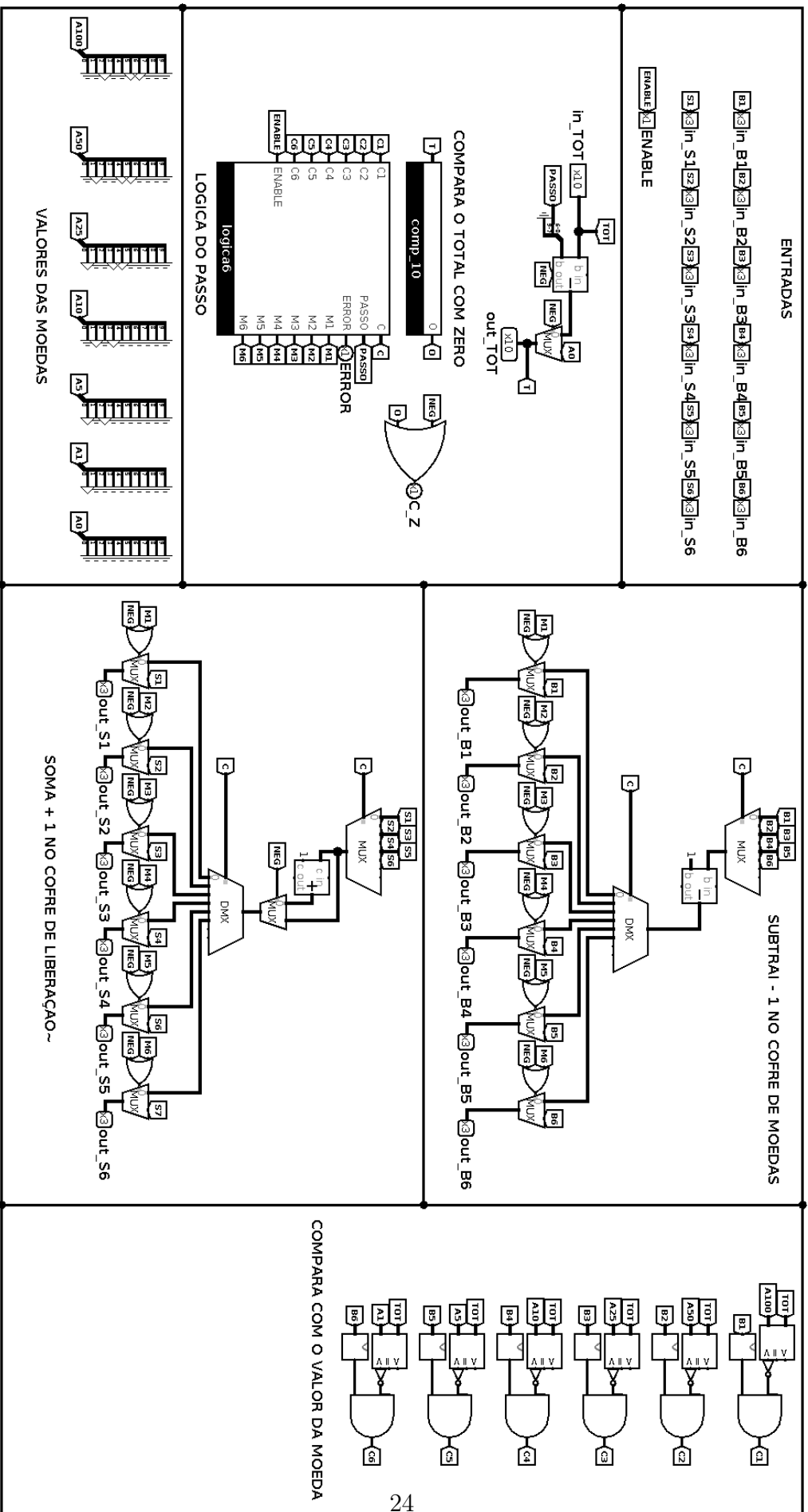


Figura B.1: Circuito Subtrator

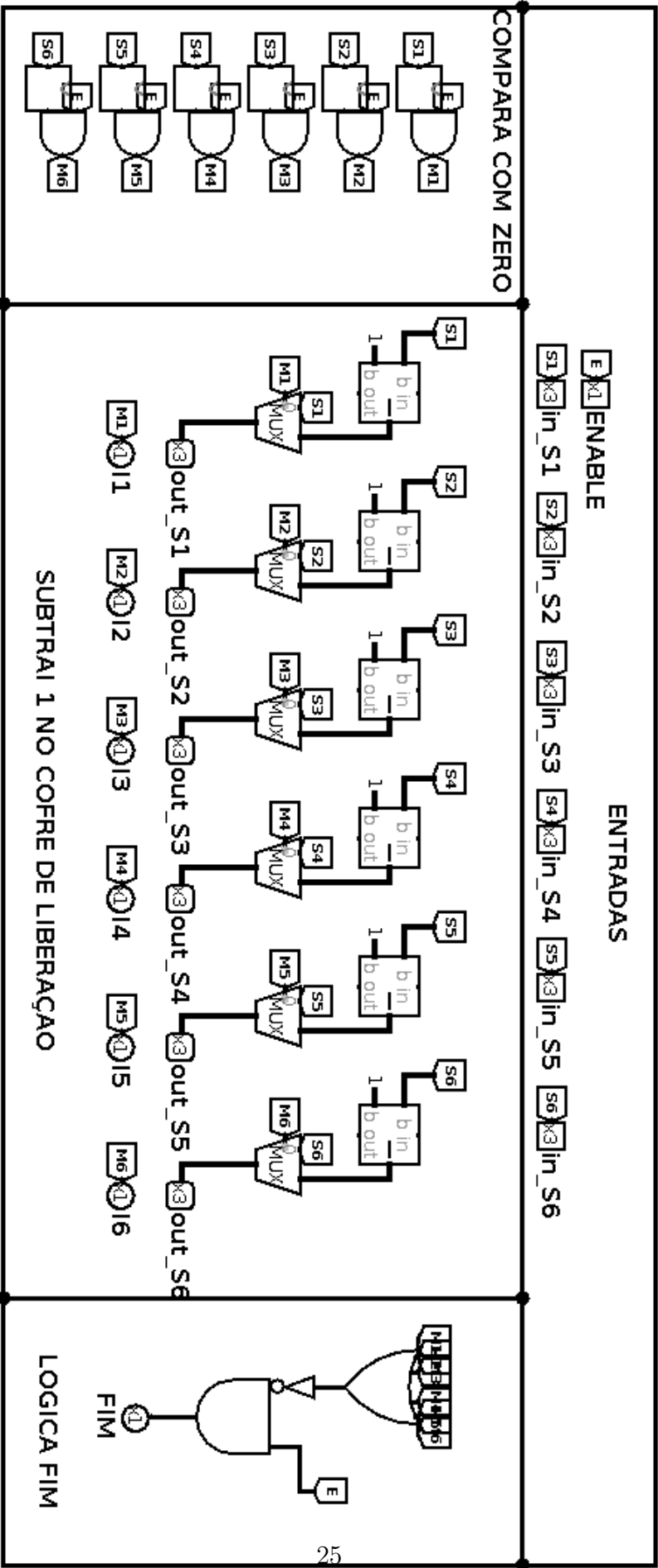


Figura B.2: Circuito Liberador

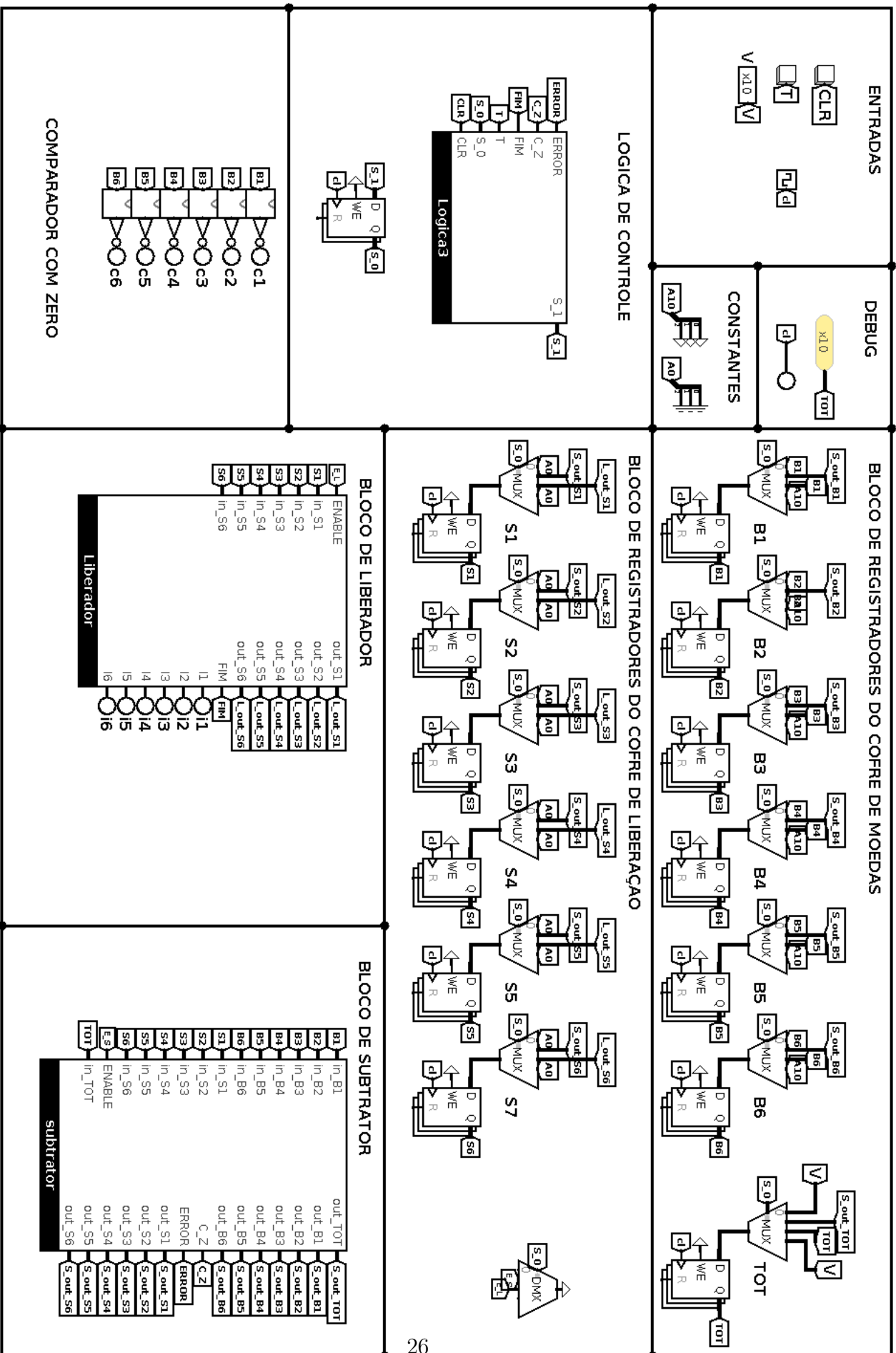


Figura B.3: Máquina de troco